TASK 03

API Security Risk Analysis

1. Objective

To learn basic API security testing using a public sample API.

2. Tools Used

- Google Chrome
- JSONPlaceholder Public Test API

3. API Endpoints Tested

Endpoint 1: https://jsonplaceholder.typicode.com/posts/1

Observation: The API returned sample post information, including userId, id, title, and body.

Endpoint 2: https://jsonplaceholder.typicode.com/users/1

Observation: The API returned sample user information, including name, email, and address fields.

4. Initial Result

Both API endpoints returned sample data successfully.

5. Security Assessment Status

Authentication, authorization, rate limiting, and input validation have not yet been verified.

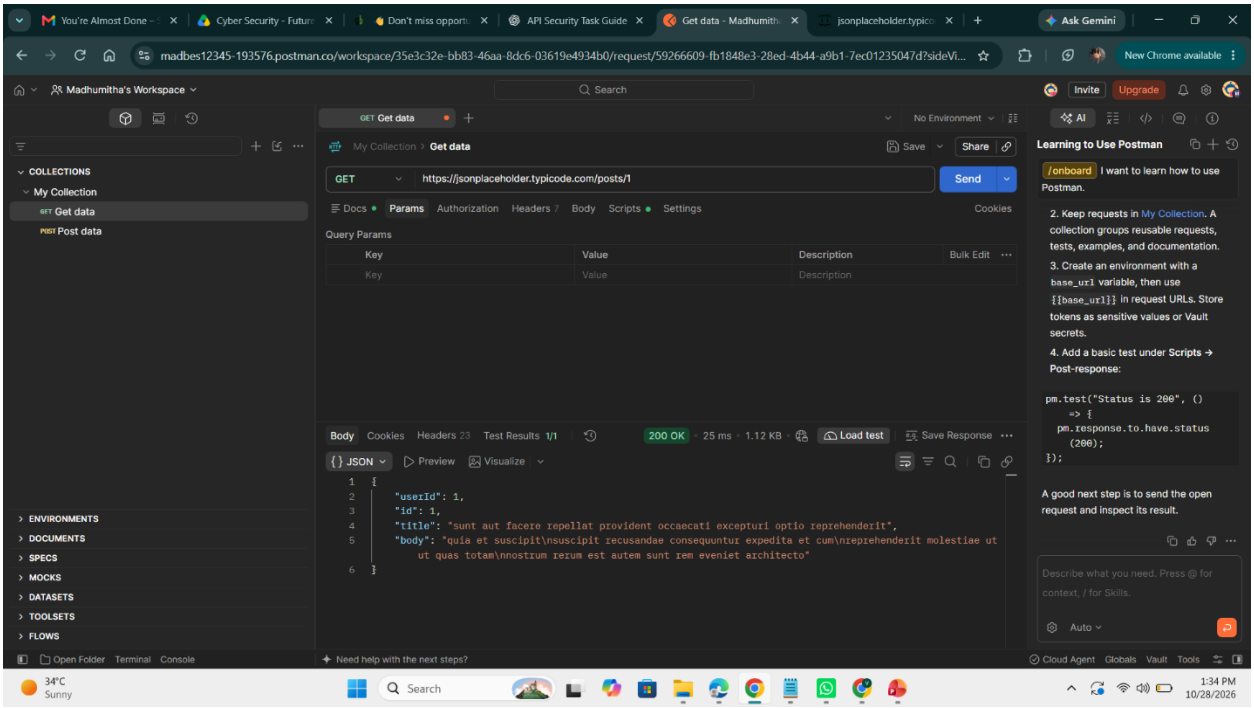
6. Additional Observation

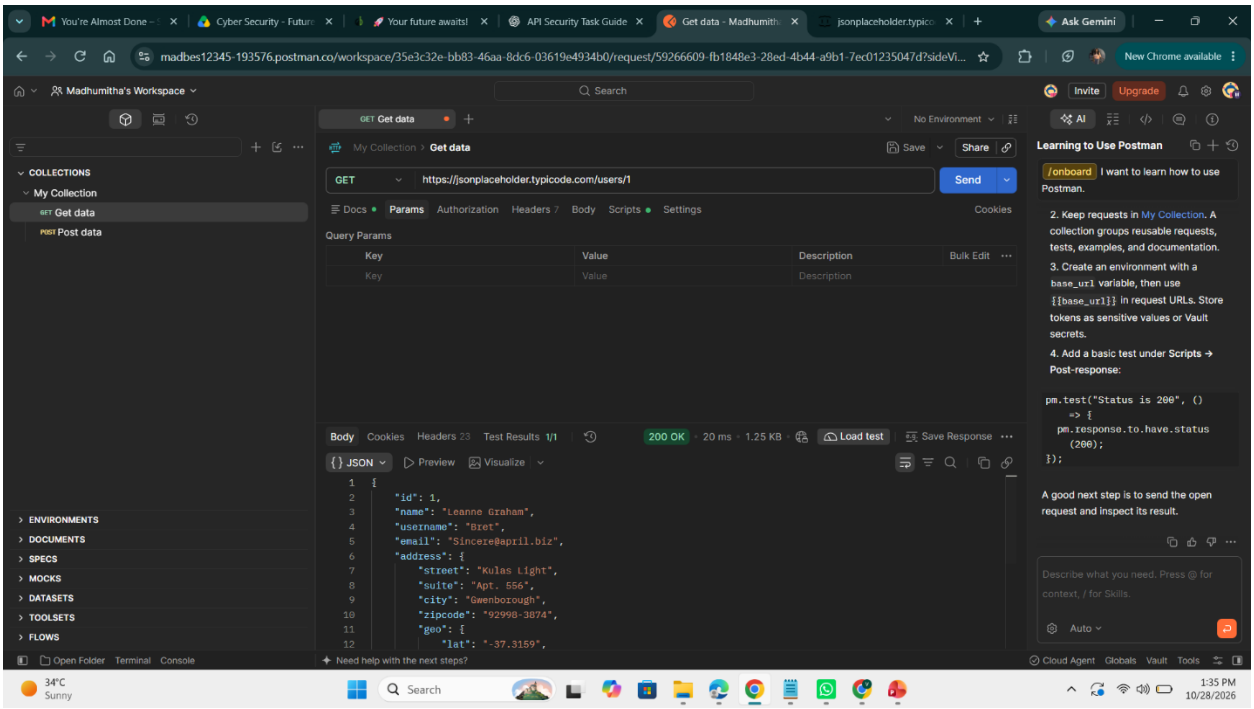
Test performed: Requested a post using a non-existing sample ID.

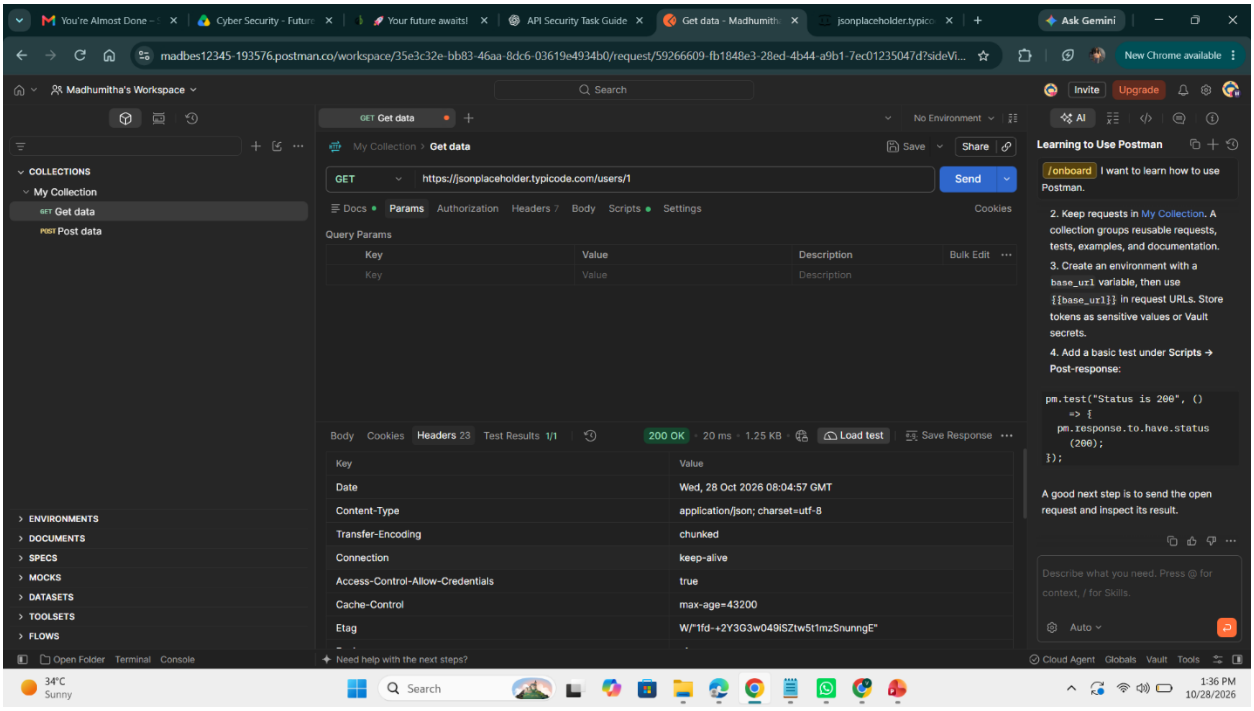
Endpoint: /posts/999999

Observation: The API returned an empty response.

Result: No post data was returned for this ID. This observation alone does not indicate a security vulnerability.


madbes12345-193576.postman.co/workspace/35e3c32e-bb83-46aa-8dc6-03619e4934b0/request/59266609-fb1848e3-28ed-4b44-a9b1-7ec01235047d?sideVi...
Madhumitha's Workspace
GET Get data
My Collection > Get data
GET
https://jsonplaceholder.typicode.com/posts/1
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Query Params
Key Value Description Bulk Edit
Body Cookies Headers 23 Test Results 1/1
200 OK 25 ms 1.12 KB
Load test
Save Response
{ } JSON Preview Visualize
{
"userId": 1,
"id": 1,
"title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
"body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut
ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
COLLECTIONS
My Collection
GET Get data
POST Post data
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Learning to Use Postman
/onboard I want to learn how to use Postman.
2. Keep requests in My Collection. A collection groups reusable requests, tests, examples, and documentation.
3. Create an environment with a base_url variable, then use {{base_url}} in request URLs. Store tokens as sensitive values or Vault secrets.
4. Add a basic test under Scripts → Post-response:
pm.test("Status is 200", () => {
pm.response.to.have.status(200);
});
A good next step is to send the open request and inspect its result.
Describe what you need. Press @ for context, / for Skills.
Auto
Need help with the next steps?
Cloud Agent Globals Vault Tools
34°C Sunny
1:34 PM 10/28/2026


madbes12345-193576.postman.co/workspace/35e3c32e-bb83-46aa-8dc6-03619e4934b0/request/59266609-fb1848e3-28ed-4b44-a9b1-7ec01235047d?sideVi...
Madhumitha's Workspace
GET Get data
My Collection > Get data
GET
https://jsonplaceholder.typicode.com/users/1
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Query Params
Key Value Description Bulk Edit
Body Cookies Headers 23 Test Results 1/1
200 OK 20 ms 1.25 KB
Load test
Save Response
{ } JSON Preview Visualize
{
"id": 1,
"name": "Leanne Graham",
"username": "Bret",
"email": "Sincere@april.biz",
"address": {
"street": "Kulas Light",
"suite": "Apt. 556",
"city": "Gwenborough",
"zipcode": "92998-3874",
"geo": {
"lat": "-37.3159",
COLLECTIONS
My Collection
GET Get data
POST Post data
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Learning to Use Postman
/onboard I want to learn how to use Postman.
2. Keep requests in My Collection. A collection groups reusable requests, tests, examples, and documentation.
3. Create an environment with a base_url variable, then use {{base_url}} in request URLs. Store tokens as sensitive values or Vault secrets.
4. Add a basic test under Scripts → Post-response:
pm.test("Status is 200", () => {
pm.response.to.have.status(200);
});
A good next step is to send the open request and inspect its result.
Describe what you need. Press @ for context, / for Skills.
Auto
Need help with the next steps?
Cloud Agent Globals Vault Tools
34°C Sunny
1:35 PM 10/28/2026



7. Response Header Inspection

API Tested: https://jsonplaceholder.typicode.com/users/1

HTTP Method: GET

Observed Status Code: 200 OK

Observed Headers:

- Content-Type: application/json; charset=utf-8
- Transfer-Encoding: chunked
- Connection: keep-alive
- Cache-Control: max-age=43200

Observation: The API successfully returned sample user data. Response headers were inspected using Postman.

Security Assessment: The observed response confirms successful communication. It does not, by itself, establish whether authentication, authorization, or other security controls are correctly implemented.

Recommendation: Production APIs should use suitable security controls based on the sensitivity of their data and intended access requirements.

8. Authentication Assessment

API: JSONPlaceholder – /users/1

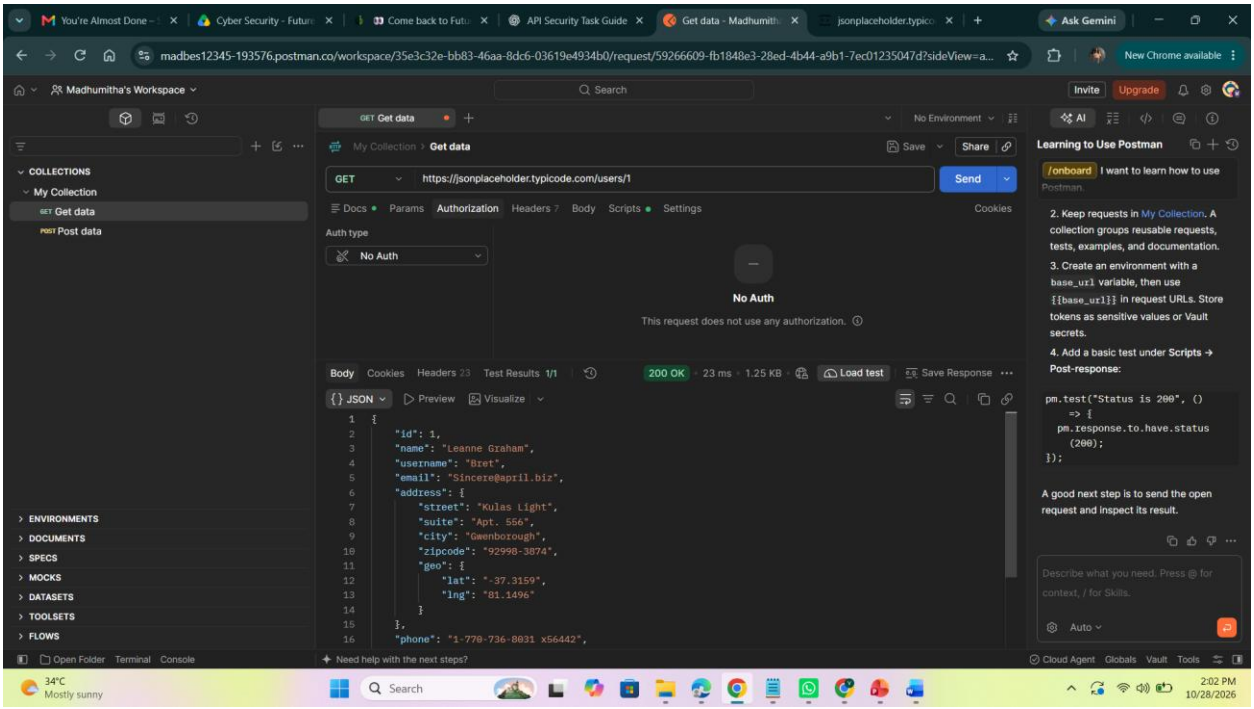
Method: GET

Observation: The request was configured with No Auth in Postman. The public demo API returned sample user data successfully.

Risk: Authentication enforcement could not be assessed using this public demo endpoint alone.

Severity: Not determined.

Recommendation: Production APIs containing private or sensitive information should require appropriate authentication and verify access permissions.



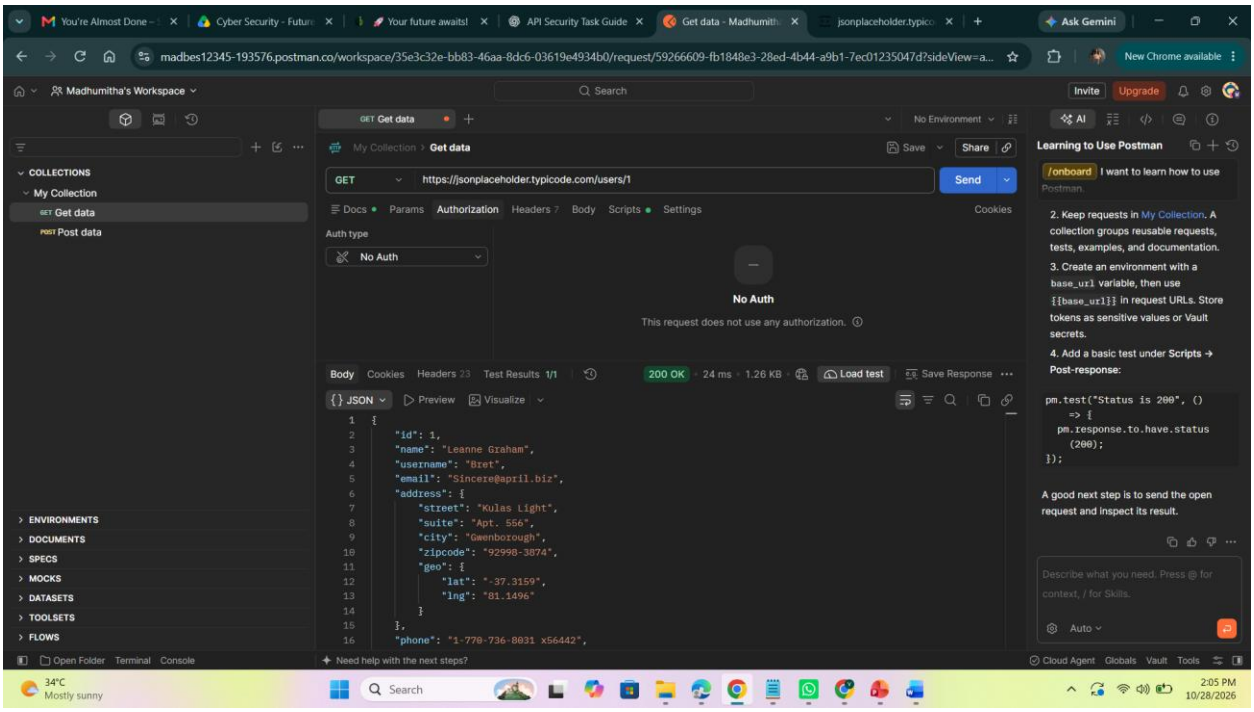
9. Data Exposure Assessment

Endpoint: GET /users/1

Observation: The public demo API returns sample user information, including name, email, address, phone, and company fields.

Risk status: No confirmed vulnerability identified from this observation alone.

Recommendation: Production APIs should return only the information required by the application and authorized user.



10. Authentication Assessment

Observation: The Postman request was configured with "No Auth" and the public JSONPlaceholder demo endpoint returned a successful response.

Risk assessment: This does not confirm an authentication vulnerability because JSONPlaceholder is a public demonstration API intended to return sample data.

Recommendation: Real applications should enforce authentication on endpoints that contain protected information and apply authorization checks for each user.

Severity: Informational — authentication controls on a production system were not assessed.

11. Rate Limiting Assessment

Observation: Rate limiting was not tested. No high-volume or repeated-request testing was performed.

Risk assessment: Rate-limiting effectiveness remains unverified.

Potential business impact: Without suitable limits, a production API may be vulnerable to excessive resource consumption.

Recommendation: Configure suitable rate limits, quotas, and monitoring for production APIs.

Severity: Not assessed.

12. Input and Error Handling Assessment

Endpoint tested: GET https://jsonplaceholder.typicode.com/posts/999999

Observation: The API returned an empty JSON array ([]) for a nonexistent post ID.

Risk assessment: No confirmed vulnerability was identified. An empty response for a nonexistent record can be normal behavior.

Recommendation: Production APIs should validate input, return consistent errors, and avoid exposing internal implementation details.

Severity: Informational — no confirmed vulnerability.

13. Scope and Limitations

This assessment used the public JSONPlaceholder demonstration API and read-only GET requests. The assessment did not include exploitation, authentication bypass, authorization bypass, load testing, or access to private systems. The results do not establish the security status of a production API.

14. Conclusion

The assessment documented API responses, response headers, sample data exposure, and behavior for a nonexistent resource. No confirmed security vulnerability was identified from these limited checks. Authentication enforcement, authorization, and rate limiting on a production API remain unverified.

15. Evidence Collected

- GET /posts/1 response.
- Response headers for GET /posts/1.
- GET /users/1 sample user response.